Proyecto E22: Sistema de Gestión de Flota de Transporte y Logística

Conceptos utilizados en este ejercicio

Esta práctica se hizo con el fin de integrar todas las bases que he aprendido sobre POO (Programación Orientada a Objetos), creando un sistema completo de gestión de una flota de transporte y logística donde utilicé conocimientos ya adquiridos y dominados como la persistencia de datos, estructuras de datos nativas como diccionarios, condicionales y estructuras de flujo como while y for, importación de librerías de Python, el uso de match para filtrar posibles casos que me lleven por diversos caminos, y también pude practicar y consolidar conocimiento nuevo como Clases, Subclases, Herencia, Encapsulamiento, Polimorfismo, La librería Path para utilizar una ruta específica de mis carpetas, el decorador @property y @atributo.setter para crear getters y setters de un atributo protegido respectivamente, la sobrecarga de métodos, y por último la creación de la documentación de este programa, sin embargo también me atreví a aprender a usar Git para inicializar el repositorio local y aprender a hacer commits y practicar para hacer commits atómicos, luego conecté el repositorio local con GitHub para tener un repositorio remoto y poder hacer Git Push para subir mis cambios a GitHub, y aprender a cambiar la descripción de commits antiguos y a eliminar el último commit. Lo más beneficioso de esta práctica fue entender el flujo de trabajo (workflow) de un programador moderno, el experimentar por primera vez lo que es ir construyendo código con un paradigma moderno con herramientas modernas como VS Code, además de realizar una conexión para que mi trabajo esté en la web y acceder a él desde otra pc o mi teléfono es bastante satisfactorio, y por último entender cómo ir subiendo los commits para tener un orden en el historial del programa me abre puertas para poder trabajar en equipo con compañeros o otros programadores.

Líbrerias importadas

Tanto la librería pickle como la pathlib las uso para el sistema de guardado y cargado, pickle para guardar el objeto empresa de la clase EmpresaLogistica, ese objeto es el que modifica los vehículos y es el que tiene el diccionario flota donde se almacenan los objetos. Y Pathlib es para asignar una carpeta donde se van a guardar y cargar los datos.

In [ ]:
```
import pickle
from pathlib import Path
```

Clases

Clase EmpresaLogistica

Es la clase que almacena los objetos camión, motocicleta, y furgoneta en su diccionario llamado flota, sirve como estructura de datos al ser el objeto que se guarda y se carga en el sistema de guardado y cargado. Además es el objeto que solicita las modificaciones de los objetos camión, motocicleta y furgoneta que se guarda en su diccionarios.

Atributos:

flota = {placa : objeto}

In [ ]:
```
class EmpresaLogistica:
    def __init__(self):
        # flota = {placa (str): vehiculo (obj)}
        self.flota = {}
```

Métodos

registrar_vehiculo(self, vehiculo): Este método verifica que no exista un objeto con la placa introducida en el diccionario flota para evitar sobre escribir objetos ya existentes, luego si pasa la verificación procede a acceder al diccionario flota y crea la clave con la placa introducida y añade el objeto vehiculo que se le pasó como argumento.

In [ ]:
```
def registrar_vehiculo(self, vehiculo):
    if vehiculo.placa in self.flota:
        print(f"El vehículo con placa: {vehiculo.placa} ya está registrado en la
        return
    self.flota[vehiculo.placa] = vehiculo
    print(f"Se ha registrado exitosamente el vehículo con placa: {vehiculo.placa
    return
```

buscar_vehiculo(self, placa): Este método verifica si la placa está en el diccionario flota, si no está devuelve None y si está devuelve el objeto vehículo que tiene la placa ingresada.

In [ ]:
```
def buscar_vehiculo(self, placa):
    if not placa.upper() in self.flota:
        return
    return self.flota[placa]
```

actualizar_kilometraje_vehiculo(self, placa, kilometraje_nuevo: float): Este método verifica que hayan objetos vehículo en el diccionario flota, si no hay imprime en pantalla que no hay, además verifica si está el objeto vehículo que tiene la placa que introducimos, esto lo hacemos con un condicional y el método buscar_vehiculo(self, placa), si no se consigue el vehículo se imprime en pantalla que esa placa es inválida porque no le pertenece a ningún objeto vehículo del diccionario flota. En caso contrario, se obtiene el objeto vehículo, se utiliza el setter de kilometraje para acceder a ese atributo protegido del objeto vehiculo para que haga las verificaciones necesarios con respecto al kilometraje nuevo que se le está asignando.

In [ ]:
```
def actualizar_kilometraje_vehiculo(self, placa, kilometraje_nuevo: float):
    if not self.flota:
        print("No hay vehículos en la flota")
        return
```

```
    if self.buscar_vehiculo(placa) is None:
        print("Introduce una placa válida")
        return
    self.buscar_vehiculo(placa).kilometraje = kilometraje_nuevo
    return
```

calcular_costo_total_mantenimiento(self): Este método verifica que hayan objetos vehículo en el diccionario flota, si no hay imprime en pantalla que no hay, luego crea la variable local costo_total_mantenimiento y la inicializa con el valor float 0.0, para después recorrer con un for los objetos vehiculo del diccionario flota y utilizar el método calcular_mantenimiento(self) del objeto vehiculo, para que calcule su mantenimiento dependiendo del objeto en cuestión e ir añadiéndole ese valor a la variable costo_total_mantenimiento en cada iteración, por último imprime el costo total del mantenimiento de la flota en pantalla.

In [ ]:
```
def calcular_costo_total_mantenimiento(self):
    if not self.flota:
        print("No hay vehículos en la flota")
        return
    costo_total_mantenimiento = 0.0
    for vehiculo in self.flota.values():
        costo_total_mantenimiento += vehiculo.calcular_mantenimiento()
    print(f"Costo total del mantenimiento de la flota: {costo_total_mantenimient
    return
```

generar_reporte(self): Este método verifica que hayan objetos vehículo en el diccionario flota, si no hay imprime en pantalla que no hay, después imprime el mensaje flota actual haciendo enfásis que son todos los vehículos de la flota, para luego recorrer los objetos vehiculo en el diccionario flota e imprimir esos objetos aprovechando el método **str** de los objetos vehiculo para que ellos impriman sus atributos protegidos.

In [ ]:
```
def generar_reporte(self):
    if not self.flota:
        print("No hay vehículos en la flota")
        return
    print("Flota actual")
    for vehiculo in self.flota.values():
        print(vehiculo)
    return
```

Clase Vehiculo

Es la clase que define lo que es un vehículo genérico en el programa, se utiliza como clase padre de las subclases Camion, Furgoneta y Motocicleta, ya que ellas son tipos de vehículos.

Atributos:

placa (protegido)

marca (protegido)

kilometraje (tipo de dato: float) (protegido)

costo_base_mantenimiento (tipo de dato: float) (protegido)

In [ ]:
```
class Vehiculo:
    def __init__(self, placa, marca, kilometraje : float, costo_base_mantenimien
        self._placa = placa.upper()
        self._marca =  marca
        self._kilometraje = kilometraje
        self._costo_base_mantenimiento = costo_base_mantenimiento
```

Métodos

kilometraje (Getter): Es el método al que se le coloca el decorador @property para convertirlo en un método especial que sirve para acceder al atributo protegido kilometraje.

In [ ]:
```
@property
def kilometraje(self):
    return self._kilometraje
```

kilometraje (Setter): Es el método al que se le coloca @kilometraje.setter para definirlo como setter del método getter del mismo nombre al que se le colocó el decorador @property, este método verifica que el kilometraje_nuevo sea mayor al que ya tiene el objeto vehiculo, si no lo es imprime un mensaje avisando, en el caso en que si sea mayor se le asigna kilometraje_nuevo el atributo protegido kilometro.

In [ ]:
```
@kilometraje.setter
def kilometraje(self, kilometraje_nuevo : float):
    if kilometraje_nuevo <= self._kilometraje:
        print(f"Introduce un kilometraje mayor que el actual: {self._kilometraje
        return
    self._kilometraje = kilometraje_nuevo
    print(f"Se ha cambiado exitosamente el kilometraje a {self._kilometraje}")
    return
```

costo_base_mantenimiento (Getter): Es el método al que se le coloca el decorador @property para convertirlo en un método especial que sirve para acceder al atributo protegido costo_base_mantenimiento.

In [ ]:
```
@property
def costo_base_mantenimiento(self):
    return self._costo_base_mantenimiento
```

calcular_mantenimiento(self): Es el método que devuelve el resultado del getter costo_base_mantenimiento.

In [ ]:
```
def calcular_mantenimiento(self):
    return self.costo_base_mantenimiento
```

placa (getter): Es el método al que se le coloca el decorador @property para convertirlo en un método especial que sirve para acceder al atributo protegido placa.

In [ ]:
```
@property
def placa(self):
```

```
    return self._placa
```

str(self): Es el método especial que se activa al imprimir directamente el objeto vehiculo por la función print(), devuelve lo que se va a mostrar en pantalla. En este caso información de los atributos protegidos del objeto vehiculo.

In [ ]:
```
def __str__(self):
    return f"Placa: {self._placa} Marca: {self._marca} Kilometraje: {self._kilom
```

Subclase Camion

La subclase Camion crea objetos del tipo vehículo al ser una clase hija de la clase Vehiculo, los camiones son un tipo de vehículos diseñados para trasladar grandes cantidades de material, por ello tienen un atributo único llamado capacidad_toneladas.

Atributos heredados:

placa (protegido)

marca (protegido)

kilometraje (tipo de dato: float) (protegido)

costo_base_mantenimiento (tipo de dato: float) (protegido)

Atributos:

capacidad_toneladas (tipo de dato: float) (protegido)

In [ ]:
```
class Camion(Vehiculo):
    def __init__(self, placa, marca, kilometraje : float, costo_base_mantenimien
        super().__init__(placa, marca, kilometraje, costo_base_mantenimiento)
        self._capacidad_toneladas = capacidad_toneladas
```

Métodos

calcular_mantenimiento(self): Es el método que devuelve el resultado del getter costo_base_mantenimiento, pero al ser un objeto de la subclase Camion se le incrementa el costo del mantenimiento un 15% por cada tonelada que tenga el vehículo. Este método devuelve el valor total del costo del mantenimiento de este objeto camion.

In [ ]:
```
def calcular_mantenimiento(self):
    return self.costo_base_mantenimiento + (0.15 * self._capacidad_toneladas)
```

str(self): Es el método especial que se activa al imprimir directamente el objeto camion por la función print(), devuelve lo que se va a mostrar en pantalla. En este caso información de los atributos protegidos del objeto camion.

In [ ]:
```
def __str__(self):
    return  f"Placa: {self._placa} Marca: {self._marca} Kilometraje: {self._kilo
```

Subclase Furgoneta

La subclase Furgoneta crea objetos del tipo vehículo al ser una clase hija de la clase Vehiculo, las furgonetas son un tipo de vehículos que sirve para transportar alimentos sensibles a la temperatura, por ello tiene un atributo único llamado es_refrigerada que nos indica si puede llevar ese tipo de alimentos o no.

Atributos heredados:

placa (protegido)

marca (protegido)

kilometraje (tipo de dato: float) (protegido)

costo_base_mantenimiento (tipo de dato: float) (protegido)

Atributos:

es_refrigerada (tipo de dato: bool) (protegido)

In [ ]:
```
class Furgoneta(Vehiculo):
    def __init__(self, placa, marca, kilometraje : float, costo_base_mantenimien
        super().__init__(placa, marca, kilometraje, costo_base_mantenimiento)
        self._es_refrigerada = es_refrigerada
```

Métodos

calcular_mantenimiento(self): Es el método que devuelve el resultado del getter costo_base_mantenimiento, pero al ser un objeto de la subclase Furgoneta se le incrementa el costo del mantenimiento en 200.0 si el objeto furgoneta es refrigerada, si no lo es, no se le incrementa nada y devuelve el resultado del getter costo_base_mantenimiento. Este método devuelve el valor total del costo del mantenimiento de este objeto furgoneta.

In [ ]:
```
def calcular_mantenimiento(self):
    if self._es_refrigerada:
         return self.costo_base_mantenimiento + 200.0
    else:
        return self.costo_base_mantenimiento
```

str(self): Es el método especial que se activa al imprimir directamente el objeto furgoneta por la función print(), devuelve lo que se va a mostrar en pantalla. En este caso se utiliza un condicional para verificar si es una furgoneta refrigerada y cambiar el mensaje que se va a imprimir en pantalla

In [ ]:
```
def __str__(self):
    if self._es_refrigerada:
        return f"Placa: {self._placa} Marca: {self._marca} Kilometraje: {self._k
    else:
        return f"Placa: {self._placa} Marca: {self._marca} Kilometraje: {self._k
```

Subclase Motocicleta

La subclase Motocicleta crea objetos del tipo vehículo al ser una clase hija de la clase Vehiculo, las motocicletas son un tipo de vehículos que sirve para transportar personas, por ello tiene un atributo único llamado cilindrada que nos indica la potencia de la moto.

Atributos heredados:

placa (protegido)

marca (protegido)

kilometraje (tipo de dato: float) (protegido)

costo_base_mantenimiento (tipo de dato: float) (protegido)

Atributos:

cilindrada (tipo de dato: int) (protegido)

In [ ]:
```
def __init__(self, placa, marca, kilometraje : float, costo_base_mantenimiento :
    super().__init__(placa, marca, kilometraje, costo_base_mantenimiento)
    self._cilindrada = cilindrada
```

Métodos

calcular_mantenimiento(self): Es el método que devuelve el resultado del getter costo_base_mantenimiento, pero al ser un objeto de la subclase Motocicleta se le incrementa el costo del mantenimiento en 20% si el objeto motocicleta tiene una cilindrada mayor a 200, si no lo es, no se le incrementa nada y devuelve el resultado del getter costo_base_mantenimiento. Este método devuelve el valor total del costo del mantenimiento de este objeto motocicleta.

In [ ]:
```
def calcular_mantenimiento(self):
    if self._cilindrada > 200:
        return self.costo_base_mantenimiento + (0.20 * self.costo_base_mantenimi
    return self.costo_base_mantenimiento
```

str(self): Es el método especial que se activa al imprimir directamente el objeto vehiculo por la función print(), devuelve lo que se va a mostrar en pantalla. En este caso se utiliza un condicional para verificar si es una furgoneta refrigerada y cambiar el mensaje que se va a imprimir en pantalla

In [ ]:
```
def __str__(self):
    return  f"Placa: {self._placa} Marca: {self._marca} Kilometraje: {self._kilo
```

Funciones

cargar_datos(ruta): Esta función se encarga de cargar el objeto en la ruta especificada, devuelve un objeto por defecto de la clase EmpresaLogistica en dado caso no se encuentre al archivo a cargar o esté vació o corrupto.

In [ ]:
```python
def cargar_datos(ruta):
    try:
        with open(ruta, "rb") as archivo:
            return pickle.load(archivo)
    except (FileNotFoundError, pickle.PickleError):
        return EmpresaLogistica()
```

guardar_datos(ruta, datos): Esta función se encarga de guardar el objeto en la ruta especificada, el objeto que guarda es de la clase EmpresaLogistica, si no se encuentra el archivo se crea y se guarda el objeto ahí, y además imprime en pantalla que se ha guardado los datos.

In [ ]:
```python
def guardar_datos(ruta, datos):
    with open(ruta, "wb") as archivo:
        pickle.dump(datos, archivo)
    print("Datos guardados exitosamente")
    return
```

CÓDIGO DEL PROGRAMA

In [ ]:
```python
#Clases

class Vehiculo:
    def __init__(self, placa, marca, kilometraje : float, costo_base_mantenimien
        self._placa = placa.upper()
        self._marca =  marca
        self._kilometraje = kilometraje
        self._costo_base_mantenimiento = costo_base_mantenimiento

    @property
    def kilometraje(self):
        return self._kilometraje

    @kilometraje.setter
    def kilometraje(self, kilometraje_nuevo : float):
        if kilometraje_nuevo <= self._kilometraje:
            print(f"Introduce un kilometraje mayor que el actual: {self._kilomet
            return
        self._kilometraje = kilometraje_nuevo
        print(f"Se ha cambiado exitosamente el kilometraje a {self._kilometraje}
        return

    @property
    def costo_base_mantenimiento(self):
        return self._costo_base_mantenimiento

    @costo_base_mantenimiento.setter
    def costo_base_mantenimiento(self, nuevo_costo_base_mantenimiento : float):
        if nuevo_costo_base_mantenimiento <= 0:
            print("Introduce un nuevo costo base del mantenimiento mayor a 0")
            return
        self._costo_base_mantenimiento = nuevo_costo_base_mantenimiento
        print(f"Se ha cambiado exitosamente el costo base del mantenimiento a {s
        return

    def calcular_mantenimiento(self):
```

```
        return self.costo_base_mantenimiento

    @property
    def placa(self):
        return self._placa

    def __str__(self):
        return f"Placa: {self._placa} Marca: {self._marca} Kilometraje: {self._k

class Camion(Vehiculo):
    def __init__(self, placa, marca, kilometraje : float, costo_base_mantenimien
        super().__init__(placa, marca, kilometraje, costo_base_mantenimiento)
        self._capacidad_toneladas = capacidad_toneladas

    def calcular_mantenimiento(self):
        return self.costo_base_mantenimiento + (0.15 * self._capacidad_toneladas

    def __str__(self):
        return  f"Placa: {self._placa} Marca: {self._marca} Kilometraje: {self._

class Furgoneta(Vehiculo):
    def __init__(self, placa, marca, kilometraje : float, costo_base_mantenimien
        super().__init__(placa, marca, kilometraje, costo_base_mantenimiento)
        self._es_refrigerada = es_refrigerada

    def calcular_mantenimiento(self):
        if self._es_refrigerada:
            return self.costo_base_mantenimiento + 200.0
        else:
            return self.costo_base_mantenimiento

    def __str__(self):
        if self._es_refrigerada:
            return f"Placa: {self._placa} Marca: {self._marca} Kilometraje: {sel
        else:
            return f"Placa: {self._placa} Marca: {self._marca} Kilometraje: {sel

class Motocicleta(Vehiculo):
    def __init__(self, placa, marca, kilometraje : float, costo_base_mantenimien
        super().__init__(placa, marca, kilometraje, costo_base_mantenimiento)
        self._cilindrada = cilindrada

    def calcular_mantenimiento(self):
        if self._cilindrada > 200:
            return self.costo_base_mantenimiento + (0.20 * self.costo_base_mante
        return self.costo_base_mantenimiento

    def __str__(self):
        return  f"Placa: {self._placa} Marca: {self._marca} Kilometraje: {self._

class EmpresaLogistica:
    def __init__(self):
        # flota = {placa (str): vehiculo (obj)}
        self.flota = {}

    def registrar_vehiculo(self, vehiculo):
        if vehiculo.placa in self.flota:
            print(f"El vehículo con placa: {vehiculo.placa} ya está registrado e
            return
        self.flota[vehiculo.placa] = vehiculo
```

```
        print(f"Se ha registrado exitosamente el vehículo con placa: {vehiculo.p
        return

    def buscar_vehiculo(self, placa):
        if not placa.upper() in self.flota:
            return
        return self.flota[placa]

    def actualizar_kilometraje_vehiculo(self, placa, kilometraje_nuevo: float):
        if not self.flota:
            print("No hay vehículos en la flota")
            return
        if self.buscar_vehiculo(placa) is None:
            print("Introduce una placa válida")
            return
        self.buscar_vehiculo(placa).kilometraje = kilometraje_nuevo
        return

    def calcular_costo_total_mantenimiento(self):
        if not self.flota:
            print("No hay vehículos en la flota")
            return
        costo_total_mantenimiento = 0.0
        for vehiculo in self.flota.values():
            costo_total_mantenimiento += vehiculo.calcular_mantenimiento()
        print(f"Costo total del mantenimiento de la flota: {costo_total_mantenim
        return

    def generar_reporte(self):
        if not self.flota:
            print("No hay vehículos en la flota")
            return
        print("Flota actual")
        for vehiculo in self.flota.values():
            print(vehiculo)
        return
```

Menú del programa

En el menú del programa se pueden elegir las opciones que se muestran en pantalla, además se introducen los datos para poder realizar las modificaciones que haya elegido el usuario y se verifica que sean correctos y cumplan con todos los requisitos, si algo sale mal en la introducción de los datos se vuelven a pedir, y por último se pueden guardar los datos que se hayan modificado y puedes salir del programa.

Flujo del programa

El flujo del programa empieza cargando los datos de la ruta indicada, si no se encuentra el archivo, se crea un objeto de la clase EmpresaLogistica para poder gestionar una flota nueva, luego se abre el menú, el usuario puede hacer todas las modificaciones que desee y por último guardar y salir.

In [ ]:
```
import pickle
from pathlib import Path
```

```python
#Funciones

def menu():
    while True:
        print("\n---Gestión de flota---")
        print("1. Registrar vehículo")
        print("2. Buscar vehículo")
        print("3. Actualizar kilometraje de un vehículo")
        print("4. Calcular costo total del mantenimiento")
        print("5. Guardar cambios")
        print("6. Generar reporte")
        print("7. Salir")

        try:
            opcion = int(input("Introduce la opción que vas a elegir: "))
        except ValueError:
            print("Tienes que introducir uno de los números de las opciones que
            continue

        match opcion:
            case 1:
                while True:
                    print("1. Registrar un Camion")
                    print("2. Registrar una Furgoneta")
                    print("3. Registrar una Motocicleta")
                    try:
                        opcion_2 = int(input("Introduce la opción que vas a eleg
                    except ValueError:
                        print("Tienes que introducir uno de los números de las o
                        continue
                    match opcion_2:
                        case 1:
                            placa = input("Introduce la placa del camión: ")
                            marca = input("Introduce la marca del camión: ")
                            try:
                                kilometraje = float(input("Introduce el kilometr
                            except ValueError:
                                print("Introduce un número como kilometraje")
                                continue
                            if kilometraje <= 0.0:
                                print("El kilometraje tiene que ser mayor o igua
                                continue
                            try:
                                costo_base_mantenimiento = float(input("Introduc
                            except ValueError:
                                print("Introduce un número como costo base del m
                                continue
                            if costo_base_mantenimiento < 0.0:
                                print("Introduce un costo base del mantenimiento
                                continue
                            try:
                                capacidad_toneladas = float(input("Introduce la
                            except ValueError:
                                print("Introduce un número como capacidad de ton
                                continue
                            if capacidad_toneladas < 0.0:
                                print("Introduce una capacidad de toneladas mayo
                                return
                            vehiculo = Camion(placa, marca, kilometraje, costo_b
                            empresa.registrar_vehiculo(vehiculo)
```

```
                        break
                    case 2:
                        placa = input("Introduce la placa de la furgoneta: "
                        marca = input("Introduce la marca de la furgoneta: "
                        try:
                            kilometraje = float(input("Introduce el kilometr
                        except ValueError:
                            print("Introduce un número como kilometraje")
                            continue
                        if kilometraje <= 0.0:
                            print("El kilometraje tiene que ser mayor o igua
                            continue
                        try:
                            costo_base_mantenimiento = float(input("Introduc
                        except ValueError:
                            print("Introduce un número como costo base del m
                            continue
                        if costo_base_mantenimiento < 0.0:
                            print("Introduce un costo base del mantenimiento
                            continue
                        opcion_furgoneta = input("¿Es refrigerada? ")
                        while True:
                            match opcion_furgoneta:
                                case "Si":
                                    es_refrigerada = True
                                    break
                                case "SI":
                                    es_refrigerada = True
                                    break
                                case"sI":
                                    es_refrigerada = True
                                    break
                                case "si":
                                    es_refrigerada = True
                                    break
                                case "No":
                                    es_refrigerada = False
                                    break
                                case "NO":
                                    es_refrigerada = False
                                    break
                                case "nO":
                                    es_refrigerada = False
                                    break
                                case "no":
                                    es_refrigerada = False
                                    break
                                case _:
                                    print("Introduce si o no")
                                    continue
                        vehiculo = Furgoneta(placa, marca, kilometraje, cost
                        empresa.registrar_vehiculo(vehiculo)
                        break
                    case 3:
                        placa = input("Introduce la placa de la motocicleta:
                        marca = input("Introduce la marca de la motocicleta:
                        try:
                            kilometraje = float(input("Introduce el kilometr
                        except ValueError:
                            print("Introduce un número como kilometraje")
```

```
                                continue
                            if kilometraje <= 0.0:
                                print("El kilometraje tiene que ser mayor o igua
                                continue
                            try:
                                costo_base_mantenimiento = float(input("Introduc
                            except ValueError:
                                print("Introduce un número como costo base del m
                                continue
                            if costo_base_mantenimiento < 0.0:
                                print("Introduce un costo base del mantenimiento
                                continue
                            try:
                                cilindrada = int(input("Introduce la cilindrada
                            except ValueError:
                                print("Introduce un número entero como cilindrad
                                continue
                            if cilindrada <= 0:
                                print("Introduce una cilindrada mayor a 0")
                                continue
                            vehiculo = Motocicleta(placa, marca, kilometraje, co
                            empresa.registrar_vehiculo(vehiculo)
                            break
                        case _:
                            print("Elige una de las opciones que aparece en el m
                case 2:
                    placa = input("Introduce la placa del vehículo: ")
                    if empresa.buscar_vehiculo(placa) is None:
                        print(f"No hay un vehículo registrado con la placa {placa} e
                        continue
                    print(empresa.buscar_vehiculo(placa))
                case 3:
                    placa = input("Introduce la placa del vehículo: ")
                    try:
                        kilometraje = float(input("Introduce el kilometraje del vehí
                    except ValueError:
                        print("Introduce un número como kilometraje")
                        continue
                    empresa.actualizar_kilometraje_vehiculo(placa, kilometraje)
                case 4:
                    empresa.calcular_costo_total_mantenimiento()
                case 5:
                    guardar_datos(ruta, empresa)
                case 6:
                    empresa.generar_reporte()
                case 7:
                    print("Saliendo...")
                    break
                case _:
                    print("Elige una de las opciones que aparece en el menú")
    return

def cargar_datos(ruta):
    try:
        with open(ruta, "rb") as archivo:
            return pickle.load(archivo)
    except (FileNotFoundError, pickle.PickleError):
        return EmpresaLogistica()

def guardar_datos(ruta, datos):
```

```
    with open(ruta, "wb") as archivo:
        pickle.dump(datos, archivo)
    print("Datos guardados exitosamente")
    return

#Flujo del programa

ruta_archivo = Path(__file__).resolve()
ruta_raiz = ruta_archivo.parent
ruta = ruta_raiz / "Datos" / "datos_empresa.pkl"
empresa = cargar_datos(ruta)
menu()
```

Prueba de encapsulamiento

El setter del kilometraje verifica si el kilometraje_nuevo ingresado es un float, y si es mayor que el kilometraje que tiene el objeto, si no cumple con ser mayor entonces se fuerza a un fallo y se imprime que se debe de introducir un kilometraje mayor que el actual, en la función menu() en la opción donde elijo para actualizar kilometraje verifico que kilometraje_nuevo sea un dato de tipo float, si no lo es lo imprime un error y vuelve al menú, no hace falta verificar que kilometraje_nuevo sea mayor a 0 ya que a la hora de registrar un vehiculo con registrar_vehiculo(self, vehiculo) el kilometraje ingresado tiene que ser mayor o igual a cero por ende kilometraje_nuevo también debe serlo. Luego, accede al atributo protegido kilometraje y se le asgina el kilometraje_nuevo cumpliendo así con el encapsulamiento, y se imprime en la pantalla que se ha cambiado exitosamente.

In [ ]:
```
@kilometraje.setter
def kilometraje(self, kilometraje_nuevo : float):
    if kilometraje_nuevo <= self._kilometraje:
        print(f"Introduce un kilometraje mayor que el actual: {self._kilometraje
        return
    self._kilometraje = kilometraje_nuevo
    print(f"Se ha cambiado exitosamente el kilometraje a {self._kilometraje}")
    return
```

Prueba de Polimorfismo

El mejor ejemplo de polimorfismo en este programa es la función calcular_mantenimiento(self), que heredan las clases Camion, Furgoneta y Motocicleta de la clase Vehiculo, esas tres clases anteriores hacen sobrecarga de métodos y sobre escriben su comportamiento dependiendo de cuál subclase sea.

In [ ]:
```
#Clase Vehiculo

def calcular_mantenimiento(self):
    return self.costo_base_mantenimiento

#Clase Camion

def calcular_mantenimiento(self):
    return self.costo_base_mantenimiento + (0.15 * self._capacidad_toneladas)
```

```
#Clase Furgoneta

def calcular_mantenimiento(self):
    if self._es_refrigerada:
         return self.costo_base_mantenimiento + 200.0
    else:
        return self.costo_base_mantenimiento

#Clase Motocicleta

def calcular_mantenimiento(self):
    if self._cilindrada > 200:
        return self.costo_base_mantenimiento + (0.20 * self.costo_base_mantenimi
    return self.costo_base_mantenimiento
```

Prueba de Acceso directo por Clave

En el método registrar_vehiculo(self, vehiculo) de la clase EmpresaLogistica se puede apreciar la utilidad del acceso directo de diccionarios al asignarle la placa a un objeto vehiculo, lo que permite buscarlo con precisión con el método buscar_vehiculo(self, placa) y devolver el objeto vehiculo ingresado con esa placa en el diccionario flota.

In [ ]:
```
def registrar_vehiculo(self, vehiculo):
    if vehiculo.placa in self.flota:
        print(f"El vehículo con placa: {vehiculo.placa} ya está registrado en la
        return
    self.flota[vehiculo.placa] = vehiculo
    print(f"Se ha registrado exitosamente el vehículo con placa: {vehiculo.placa
    return

def buscar_vehiculo(self, placa):
    if not placa.upper() in self.flota:
        return
    return self.flota[placa]
```

Prueba de Persistencia

Con la función de guardar_datos(ruta, datos) y cargar_datos(ruta) se realizó una prueba de persistencia exitosa que guarda el archivo en la ruta E22\Datos, desde ese ruta al iniciar el programa se carga el archivo datos_empresa.pkl, en dado caso no se encuentre se devuelve un objeto de la clase EmpresaLogistica predeterminado, es decir, una nueva sesión sin datos, dado que o no existen o se habrán dañado, y ya desde la función menu() el usuario puede guardar los datos cuando desee.

In [ ]:
```
def cargar_datos(ruta):
    try:
        with open(ruta, "rb") as archivo:
            return pickle.load(archivo)
    except (FileNotFoundError, pickle.PickleError):
        return EmpresaLogistica()

def guardar_datos(ruta, datos):
```

```
    with open(ruta, "wb") as archivo:
        pickle.dump(datos, archivo)
    print("Datos guardados exitosamente")
    return

ruta_archivo = Path(__file__).resolve()
ruta_raiz = ruta_archivo.parent
ruta = ruta_raiz / "Datos" / "datos_empresa.pkl"
empresa = cargar_datos(ruta)
menu()
```